DOOH-AR · dooh_web

Pixi2D !’ Pixi3D

Transition Specification

Version 1.0 · March 2026

1. Purpose & Scope

This document specifies the complete migration path for the dooh_web folder of the DOOH-AR-Senior-D project from its current PixiJS 2D-only rendering stack to Pixi3D, unlocking true 3D AR capabilities (billboard sprites, glTF models, perspective-correct overlays) while preserving all existing filter logic, detection pipeline, and React component architecture.

What stays the same

- React/Vite project structure and component tree
- ONNX-based building detector (Detector, useDetectionLoop)
- Camera abstraction (useCamera, Camera component)
- Geolocation gate (useGeolocation, isNearLandmark)
- FilterPicker UI component
- Filter module contract — each filter exports id, label, and draw()

What changes

- usePixiOverlay — rewired to bootstrap a Pixi3D-aware stage
- Stage layer model — effectSprite replaced by a Pixi3D Container3D layer
- Filter draw() signature — receives a Container3D instead of a flat PIXI.Graphics + textContainer pair
- package.json — adds pixi3d, pins pixi.js to v7

2. Library Versions & Compatibility

& Warning Pixi3D 2.5.0 (the current latest) targets PixiJS v5, v6, and v7. PixiJS v8 introduced breaking API changes (async init, canvas instead of view) and Pixi3D does not yet support it. Pin pixi.js to ^7.4.2 for this migration.

Package	Version / Notes
pixi.js	`^7.4.2 — must NOT upgrade to v8 while on Pixi3D 2.x`
pixi3d	`^2.5.0 — import from "pixi3d/pixi7"`
@pixi/filter-*	Any v5/v6/v7-compatible filter packages remain compatible
onnxruntime-web	No change
React / Vite	No change

Install command

```
npm install pixi.js@^7.4.2 pixi3d
```

3. Architecture: Current vs Target

3.1 Current Stage Layers (PixiJS 2D only)

Layer	Object	Purpose
0	bgSprite	`Full-frame video background (no effects)`
1	maskGfx	`PIXI.Graphics rectangle — clip mask for effectSprite`
2	effectSprite	Same video texture masked to bounding box; 2D pixel filters applied
3	decorGfx	`PIXI.Graphics cleared each frame; 2D line decorations`
4	textContainer	`PIXI.Container of PIXI.Text nodes; cleared each frame`

3.2 Target Stage Layers (Pixi3D)

Layer	Object	Purpose
0	bgSprite	Full-frame video background — unchanged from current
1	maskGfx + effectSprite	`PIXI 2D masked region for pixel-shader filters — unchanged`
2	`scene3d (Container3D)`	NEW — Pixi3D scene graph root; perspective camera anchored to bounding box
3	`decorGfx (PIXI.Graphics)`	Flat 2D vector decorations on top of 3D scene — unchanged contract
4	textContainer	`PIXI.Text labels — unchanged from current`

4. API Mapping: PIXI 2D → Pixi3D

The table below covers every Pixi2D construct currently used in the codebase and its Pixi3D equivalent.

Current (PIXI 2D)	Target (Pixi3D)	Notes
`import * as PIXI from "pixi.js"`	`import * as PIXI from "pixi.js"`	Both imports required side-by-side
`PIXI.Application({})`	`PIXI.Application({})` — unchanged	Pixi3D uses the same PIXI.Application object
`PIXI.Container`	`PIXI3D.Container3D`	Root 3D scene node; add to app.stage as usual
`PIXI.Sprite (video)`	`PIXI.Sprite (video)`	bgSprite and effectSprite stay as 2D sprites
`PIXI.Graphics (decorGfx)`	`PIXI.Graphics (decorGfx)`	2D vector decorations remain on top
`PIXI.Text / PIXI.TextStyle`	`PIXI.Text / PIXI.TextStyle`	Labels remain 2D PIXI.Text in textContainer
`n/a (2D only)`	`PIXI3D.Mesh3D.createPlane()`	Flat plane anchored to bounding box
n/a	`PIXI3D.Model.from(gltf)`	Load glTF 3D model via PIXI.Assets.load()
n/a	`PIXI3D.Sprite3D`	Billboard sprite in 3D space (always faces camera)
n/a	`PIXI3D.Camera.main`	Perspective camera; use worldToScreen / screenToWorld
n/a	`PIXI3D.Light`	Point/directional light; add to LightingEnvironment.main
`PIXI.Filter subclass`	`PIXI.Filter subclass` (unchanged)	2D pixel filters still apply to effectSprite
`app.ticker.add(fn)`	`app.ticker.add(fn)` — unchanged	Pixi3D auto-renders within the same ticker loop

5. File-by-File Changes

5.1 hooks/usePixiOverlay.js (PRIMARY CHANGE)

> **Note:** This is the only hook that needs a meaningful rewrite. All other hooks are unchanged.

Replace the current 2D-only init() body with the following structure:

```
import * as PIXI   from 'pixi.js';
import * as PIXI3D from 'pixi3d/pixi7';

function init() {
  const app = new PIXI.Application({
    view: canvas, width: video.videoWidth || 640,
    height: video.videoHeight || 480,
    antialias: true, backgroundColor: 0x000000,
  });
  appRef.current = app;

  // Layer 0+1+2: video bg + masked effect sprite (unchanged)
  const videoResource = new PIXI.VideoResource(video, { autoPlay: false });
  const videoTex = new PIXI.Texture(new PIXI.BaseTexture(videoResource));
  const bgSprite = new PIXI.Sprite(videoTex);
  bgSprite.width = app.screen.width; bgSprite.height = app.screen.height;
  app.stage.addChild(bgSprite);
  const maskGfx = new PIXI.Graphics();
  const effectSprite = new PIXI.Sprite(videoTex);
  effectSprite.mask = maskGfx;
  app.stage.addChild(maskGfx); app.stage.addChild(effectSprite);

  // Layer 3: Pixi3D scene
  const scene3d = app.stage.addChild(new PIXI3D.Container3D());

  // Layer 4+5: 2D decorations & text (unchanged)
  const decorGfx = new PIXI.Graphics();
  app.stage.addChild(decorGfx);
  const textContainer = new PIXI.Container();
  app.stage.addChild(textContainer);

  // Render loop
  app.ticker.add(() => {
    // ... pixel filter + mask update (unchanged) ...
    scene3d.removeChildren();
    if (filter.draw3d && detections.length > 0)
      filter.draw3d(scene3d, detections, time, app.screen);
    decorGfx.clear(); textContainer.removeChildren();
    if (filter.draw && detections.length > 0)
      filter.draw(decorGfx, textContainer, detections, time, app.screen);
  });
}
```

5.2 filters/*.jsx (Additive — no breaking changes)

Each filter module gains an optional draw3d() export. The existing draw() export keeps working exactly as before — legacy 2D filters require zero changes.

New optional filter API:

```
// New optional export — gives access to the Pixi3D scene Container3D
// scene3d   : PIXI3D.Container3D — scene root, cleared each frame
// detections: array of detection objects (same shape as always)
// time      : seconds (app.ticker.lastTime / 1000)
// screen    : PIXI.Rectangle (app.screen)
export function draw3d(scene3d, detections, time, screen) {
  const det = detections[0];
  const { x1, y1, x2, y2 } = det.box;

  if (cachedModel) {
    const instance = scene3d.addChild(PIXI3D.Model.from(cachedGltf));
    const cx = (x1 + x2) / 2;
    const cy = (y1 + y2) / 2;
    const worldPos = PIXI3D.Camera.main.screenToWorld(cx, cy, 5);
    instance.position.set(worldPos.x, worldPos.y, worldPos.z);
    instance.rotationQuaternion.setEulerAngles(0, time * 45, 0);
  }
}
```

Example: cyber.jsx with draw3d() addition

The existing draw() is kept verbatim. A draw3d() function is added alongside it:

```
import * as PIXI3D from 'pixi3d/pixi7';

// Keep all existing draw() code unchanged ...

export function draw3d(scene3d, detections, time, screen) {
  const { x1, y1, x2, y2 } = detections[0].box;
  const cx = (x1 + x2) / 2;
  const cy = (y1 + y2) / 2;

  // Billboard sprite — always faces the camera
  const sprite = scene3d.addChild(new PIXI3D.Sprite3D(cyberTexture));
  const wp = PIXI3D.Camera.main.screenToWorld(cx, cy - 80, 3);
  sprite.position.set(wp.x, wp.y, wp.z);
  sprite.scale.set(0.4 + Math.sin(time * 2) * 0.05);
}
```

5.3 All Other Files — No Changes Required

The following files require zero modifications:

- src/App.jsx
- src/components/camera/Camera.jsx and useCamera.js
- src/components/ai/Detector.js

- src/hooks/useDetectionLoop.js
- src/hooks/useGeolocation.js
- src/components/CameraControls.jsx
- src/components/FilterPicker.jsx
- src/components/LocationStatus.jsx
- src/util/geolocation.js
- src/filters/index.js (filters/FILTERS array)

6. Loading glTF 3D Assets

Pixi3D uses the glTF 2.0 format. Assets should be loaded once (outside the render loop) using PIXI.Assets:

```
import * as PIXI   from 'pixi.js';
import * as PIXI3D from 'pixi3d/pixi7';

// Load once — e.g. in a filter module at module scope
let buildingModelGltf = null;

export async function preload() {
  buildingModelGltf = await PIXI.Assets.load('/assets/building-marker.gltf');
}

// Then inside draw3d():
if (buildingModelGltf) {
  const model = scene3d.addChild(PIXI3D.Model.from(buildingModelGltf));
  model.position.set(worldX, worldY, worldZ);
}
```

Place .gltf and .bin files under public/assets/ so Vite serves them as static assets.

> **Warning:** scene3d.removeChildren() is called every frame. Do not add heavy glTF models inside the render loop without instancing. Either use a persistent model that you update each frame (position, rotation) instead of re-adding it, or use PIXI3D instancing for repeated meshes.

7. Camera & Coordinate System

7.1 Bridging Screen Space to 3D World Space

The YOLO detector outputs bounding boxes in pixel/canvas coordinates (x1, y1, x2, y2 relative to the PixiJS canvas). Pixi3D's Camera.main provides two helpers to cross between spaces:

Method	Use
`Camera.main.screenToWorld(x, y, depth)`	Convert a canvas pixel to a 3D world point at the given depth. Use to place 3D objects above detected buildings.
`Camera.main.worldToScreen(x3d, y3d, z3d)`	Convert a 3D world position back to a 2D screen pixel. Use to pin 2D text labels at the projected top of a 3D object.

7.2 Recommended Camera Setup

```
// After scene3d is added to the stage:
PIXI3D.Camera.main.fieldOfView = 60; // degrees — tune to match device FOV

// Keep camera at origin looking down -Z (default)
// Objects placed via screenToWorld(cx, cy, depth) will appear
// correctly positioned over the detected bounding box.
```

8. Step-by-Step Migration Checklist

1. Install updated packages: `npm install pixi.js@^7.4.2 pixi3d`
2. Verify pixi.js version is 7.x in node_modules
3. Update usePixiOverlay.js — add scene3d Container3D layer per Section 5.1. Keep all existing bgSprite / maskGfx / effectSprite / decorGfx / textContainer code intact.
4. Add scene3d pass to render loop: call filter.draw3d() if it exists; keep the filter.draw() call unchanged for backward compatibility.
5. Smoke-test existing filters: run the app, switch through all filters, confirm no visual regression. The 2D path through draw() must be identical to pre-migration.
6. Add draw3d() to one filter (e.g., cyber.jsx) using a simple Mesh3D.createPlane() or Sprite3D — confirm 3D object renders and is positioned correctly over the detected building.
7. Add public/assets/ directory; place your first .gltf model there.
8. Implement preload() in the target filter module; call it from usePixiOverlay init() using PIXI.Assets.
9. Load glTF model in draw3d(); position via Camera.main.screenToWorld().
10. Iterate filter by filter — each one can add draw3d() independently at its own pace.

9. Known Constraints & Risks

Constraint	Mitigation
Pixi3D max PixiJS v7	Pin pixi.js to ^7.4.2. Do not upgrade to v8 until Pixi3D publishes v8 support. Monitor github.com/jnsmalm/pixi3d releases
No pixi3d v8 support yet	Alternative: if v8 is required, substitute pixi3d with Three.js as an overlay canvas using the PixiJS + Three.js mixing pattern
`scene3d.removeChildren()` overhead	If a 3D model is persistent, lift it out of the loop: add once, mutate transform each frame rather than re-adding
Mobile WebGL constraints	`Pixi3D uses WebGL2 by default. Test on iOS Safari (WebGL1 fallback available). Avoid complex PBR materials`
Camera depth calibration	`screenToWorld() depth parameter is abstract. Start at depth=5 and tune empirically until overlay aligns with`
glTF asset size / load time	Keep .gltf models under 500 KB for mobile. Use Draco compression in Blender export and enable the PIXI3D Draco decoder if needed.

10. 3D Feature Roadmap (Post-Migration)

Once the baseline Pixi3D integration is stable, the following 3D AR capabilities become available:

Phase 1 — Flat 3D (immediate)

- Sprite3D billboard labels — replace PIXI.Text with perspective-scaled 3D sprites
- Mesh3D.createPlane() floor decals — project flat plane textures onto the ground at the building footprint
- Animated glTF icons — spinning logos or markers above the detected centroid

Phase 2 — Spatial AR

- glTF building annotation models — floating name cards that always face the camera (Sprite3D)
- Shadow casting — configure PIXI3D.ShadowCastingLight with a StandardPipeline
- Custom GLSL materials — subclass Material for custom shader effects on 3D geometry

Phase 3 — Advanced

- Skeletal animation — animate characters or structural walkthroughs tied to detection confidence
- Image-based lighting (IBL) — use PIXI3D.ImageBasedLighting with a Cubemap for realistic ambient light
- Multiple building instances — PIXI3D instancing for rendering many markers simultaneously at low GPU cost